

MASJAVAS AI

Prompt Instruksi Implementasi Desktop Windows Build - Electron, Express, FFmpeg, Branding, Installer

Dokumen ini siap dikirim ke programmer sebagai instruksi implementasi lengkap.

Item	Keputusan
Target	Aplikasi desktop Windows untuk MASJAVAS AI
Stack desktop	Electron + electron-builder
Frontend	React/Vite production build dari folder dist
Backend	Embedded Express server dengan dynamic localhost port
Export engine	Bundled FFmpeg + ffprobe dengan execFile array args
Storage	AppData/Roaming/MASJAVAS AI, bukan folder source code
Branding	resources/icon.ico multi-size, splash screen, installer icon
Security	GrokPI API key tidak boleh dibundle di installer

1. Konteks Sistem Saat Ini

MASJAVAS AI sudah berkembang dari single-session wizard menjadi workspace produksi video AI dengan project persistence, autosave, restore backup, ratio lock, storyboard automation, image-to-video pipeline, audio/emotion lock, preview, export REAL-MVP, dan FFmpeg final export pipeline.

Build desktop Windows harus menjaga semua fitur tersebut tetap berjalan tanpa terminal, tanpa membuka browser manual, dan tanpa menyimpan data user di folder source code.

- Frontend berjalan dengan React dan Vite.
- Backend lokal berjalan dengan Express API gateway.
- Project disimpan sebagai JSON fisik, dengan backup dan corruption guard.
- Uploads, exports, temp, dan logs menggunakan file system.
- FFmpeg export pipeline sudah tersedia dan harus dibundle untuk Windows.
- Aplikasi perlu icon, installer, splash, shortcut, dan penyimpanan data user yang aman.

2. Keputusan Arsitektur

Gunakan Electron untuk MVP desktop Windows. Jangan gunakan Tauri dulu karena pipeline backend Node/Express dan FFmpeg sudah matang di ekosistem Node. Jangan membuat shortcut browser atau sekadar membuka localhost dev server.

Area	Keputusan
Desktop shell	Electron main process
Renderer	dist/index.html hasil npm run build
Backend	Express embedded server dari main process
Port	Dynamic port di 127.0.0.1, bukan fixed 3000 untuk production desktop
API bridge	preload.js dengan contextIsolation true
Data path	app.getPath('userData')
Packaging	electron-builder NSIS installer
Icon	resources/icon.ico multi-size
FFmpeg	resources/ffmpeg/win32/ffmpeg.exe dan ffprobe.exe

3. Prompt Lengkap untuk Programmer

Salin seluruh blok instruksi berikut dan kirim ke programmer. Instruksi ini sudah disusun sebagai implementation plan lengkap.

```
TASK UTAMA - IMPLEMENTASI DESKTOP WINDOWS MASJAVAS AI

Kita akan membuat MASJAVAS AI menjadi aplikasi desktop Windows production-ready.

Gunakan pendekatan:
Electron + electron-builder + Vite production build + embedded Express backend + bundled FFmpeg.

Target akhir:
MASJAVAS AI.exe
- membuka React UI dari dist/index.html
- menjalankan backend Express embedded otomatis
- menyimpan project di AppData/Roaming
- menjalankan FFmpeg bundled
- menjaga GrokPI secret tidak bocor
- bisa dipakai user tanpa terminal
- tampil profesional dengan icon, installer, splash, dan shortcut Windows

Jangan minta approval untuk subtask kecil. Implementasikan bertahap, test setiap fase, lalu kirim laporan.
```

4. Struktur Folder Target

```
desktop/  
  main.js  
  preload.js  
  serverLauncher.js  
  
resources/  
  icon.ico  
  icon-dev.ico  
  icons/  
    icon-1024.png  
    icon-512.png  
    icon-256.png  
    icon-128.png  
    icon-64.png  
    icon-48.png  
    icon-32.png  
    icon-16.png  
  tray.ico  
  branding/  
    splash.png  
  ffmpeg/  
    win32/  
      ffmpeg.exe  
      ffprobe.exe  
  
server/  
  index.js  
  app.js  
  routes/  
  services/  
  utils/  
    ffmpegResolver.js  
    runtimePaths.js  
  
dist/  
  index.html  
  assets/
```

5. Phase A - Audit dan Refactor Entry Point Backend

Backend Express harus bisa dijalankan dalam dua mode: dev server normal dan embedded Electron server. Jangan biarkan server/index.js langsung mengunci port 3000 secara permanen dalam mode packaged desktop.

REQUIREMENT:

1. Pisahkan Express app creation dari server listen.

Buat atau refactor:

```
server/app.js  
server/index.js
```

```
server/app.js:  
- export function createApp(runtimeConfig)  
- mount routes  
- mount static uploads/exports  
- mount health check  
- apply CORS sesuai runtime
```

```
server/index.js:  
- hanya untuk dev/server mode  
- membaca PORT dari env atau default 3000  
- memanggil createApp()  
- app.listen(PORT)
```

2. Desktop Electron akan import createApp() dari server/app.js dan menjalankan app.listen(dynamicPort, "127.0.0.1").

3. Health check wajib tetap ada:
GET /api/health

Response:

```
{
```

```

"status": "ok",
"service": "masjavas-backend-proxy",
"mode": "desktop" | "dev",
"port": <number>
}

```

6. Phase B - Electron Main Process

Buat desktop/main.js

Tanggung jawab:

1. app ready
2. resolve userData path
3. start embedded Express backend
4. wait health ready
5. tampilkan splash saat backend start
6. create BrowserWindow utama
7. load dist/index.html saat packaged
8. load http://localhost:5174 saat dev desktop
9. cleanup backend process/server saat app quit

BrowserWindow security:

```

webPreferences: {
  preload: path.join(__dirname, "preload.js"),
  contextIsolation: true,
  nodeIntegration: false,
  sandbox: false
}

```

Window:

```

- title: "MASJAVAS AI"
- width: 1440
- height: 920
- minWidth: 1180
- minHeight: 760
- backgroundColor: "#020617"
- icon: resources/icon.ico

```

Contoh alur main.js:

```

const { app, BrowserWindow } = require("electron");
const path = require("path");
const { startEmbeddedServer } = require("../serverLauncher");

let mainWindow;
let backendServer;

async function createMainWindow() {
  const runtime = await startEmbeddedServer({
    userDataDir: app.getPath("userData"),
    isPackaged: app.isPackaged
  });

  mainWindow = new BrowserWindow({
    width: 1440,
    height: 920,
    minWidth: 1180,
    minHeight: 760,
    title: "MASJAVAS AI",
    backgroundColor: "#020617",
    icon: app.isPackaged
      ? path.join(process.resourcesPath, "icon.ico")
      : path.join(__dirname, "../resources/icon.ico"),
    webPreferences: {
      preload: path.join(__dirname, "preload.js"),
      contextIsolation: true,
      nodeIntegration: false
    }
  });

  mainWindow.webContents.on("did-finish-load", () => {
    mainWindow.webContents.send("masjavas:api-base-url", runtime.apiBaseUrl);
  });

  if (app.isPackaged) {
    await mainWindow.loadFile(path.join(__dirname, "../dist/index.html"));
  }
}

```

```

    } else {
      await mainWindow.loadURL("http://localhost:5174");
    }
  }
}

app.whenReady().then(createMainWindow);
app.on("before-quit", () => {
  if (backendServer && backendServer.close) backendServer.close();
});

```

7. Phase C - Server Launcher dan Dynamic Port

Buat desktop/serverLauncher.js

Requirement:

- cari available port dengan get-port
- start Express di 127.0.0.1:<dynamicPort>
- set env runtime paths
- return { port, apiBaseUrl, server }

Jangan fixed ke 3000 saat packaged desktop.
Port 3000 hanya fallback dev.

Gunakan:

npm install get-port

Contoh:

```

const getPort = require("get-port");
const { createApp } = require("../server/app");

async function startEmbeddedServer({ userDataDir, isPackaged }) {
  const port = await getPort({ port: [3000, 3001, 3002, 3010, 0] });

  const app = createApp({
    mode: isPackaged ? "desktop" : "dev",
    userDataDir
  });

  const server = await new Promise((resolve) => {
    const s = app.listen(port, "127.0.0.1", () => resolve(s));
  });

  return {
    port,
    apiBaseUrl: `http://127.0.0.1:${port}`,
    server
  };
}

module.exports = { startEmbeddedServer };

```

8. Phase D - Dynamic API Base URL di Frontend

Frontend tidak boleh hardcode http://localhost:3000 untuk packaged desktop. Gunakan bridge dari preload atau global runtime config.

Buat desktop/preload.js

```

const { contextBridge, ipcRenderer } = require("electron");

let apiBaseUrl = "http://localhost:3000";

ipcRenderer.on("masjvas:api-base-url", (_event, url) => {
  apiBaseUrl = url;
});

contextBridge.exposeInMainWorld("masjvas", {
  getApiBaseUrl: () => apiBaseUrl,
  platform: "desktop-windows"
});

Update src/services/apiBase.ts

export function getApiBaseUrl() {
  const desktopApi = window?.masjvas?.getApiBaseUrl?();
}

```

```

if (desktopApi) return desktopApi;

if (import.meta.env.VITE_API_BASE_URL) {
  return import.meta.env.VITE_API_BASE_URL;
}

return "http://localhost:3000";
}

```

Lalu semua service layer wajib memakai `getApiBaseUrl()`. Jangan hardcode `localhost:3000` di service aktif.

9. Phase E - Runtime Storage ke AppData

Production desktop tidak boleh menulis data user ke folder source code. Semua projects, uploads, exports, temp, dan logs harus pindah ke `app.getPath('userData')`.

Target Windows:
 C:\Users\<User>\AppData\Roaming\MASJAVAS AI\

Struktur:
 data/projects/
 uploads/
 exports/
 temp/
 logs/
 Buat `server/utils/runtimePaths.js`

```

function createRuntimePaths(userDataDir) {
  return {
    userDataDir,
    projectsDir: path.join(userDataDir, "data", "projects"),
    uploadsDir: path.join(userDataDir, "uploads"),
    exportsDir: path.join(userDataDir, "exports"),
    tempDir: path.join(userDataDir, "temp"),
    logsDir: path.join(userDataDir, "logs")
  };
}

```

`module.exports = { createRuntimePaths };`

- `projectRepository.js` harus membaca `projectsDir` dari runtime config/env.
- reference upload service harus menulis ke `uploadsDir`.
- `exportJobService` harus menulis ke `exportsDir` dan `tempDir`.
- Express static serving `/uploads` dan `/exports` harus menunjuk ke runtime directory.
- Saat dev biasa, fallback boleh tetap `server/data`, `server/uploads`, `server/exports`.

10. Phase F - FFmpeg Bundling dan Resolver

FFmpeg wajib dibundle agar user Windows tidak perlu install FFmpeg manual. Gunakan `execFile` dengan array args. Jangan kembali ke `exec` shell string.

Buat `server/utils/ffmpegResolver.js`

```

const path = require("path");

function getFfmpegPath() {
  if (process.env.MASJAVAS_FFmpeg_PATH) return process.env.MASJAVAS_FFmpeg_PATH;

  if (process.env.MASJAVAS_DESKTOP_PACKAGED === "true") {
    return path.join(process.resourcesPath, "ffmpeg", "win32", "ffmpeg.exe");
  }

  return "ffmpeg";
}

function getFfprobePath() {
  if (process.env.MASJAVAS_FFPROBE_PATH) return process.env.MASJAVAS_FFPROBE_PATH;

  if (process.env.MASJAVAS_DESKTOP_PACKAGED === "true") {
    return path.join(process.resourcesPath, "ffprobe", "win32", "ffprobe.exe");
  }

  return "ffprobe";
}

```

```
}  
  
module.exports = { getFfmpegPath, getFfprobePath };
```

- ffmpegService.js harus memakai getFfmpegPath() dan getFfprobePath().
- Test wajib mencakup folder install dengan spasi.
- Test filter: fit_blur, crop_center, letterbox.
- Pastikan output dimensions selalu genap untuk libx264.

11. Phase G - Branding, Icon, Splash, Installer Visual

Branding desktop Windows harus terlihat profesional. Icon bukan tambahan terakhir; icon harus masuk ke installer, taskbar, shortcut, title bar, dan Start Menu.

Asset	Lokasi	Keterangan
App icon	resources/icon.ico	Multi-size ICO: 16, 24, 32, 48, 64, 128, 256
PNG source	resources/icons/icon-1024.png	Sumber desain resolusi tinggi
Tray icon	resources/icons/tray.ico	Opsional untuk future tray/background jobs
Splash	resources/branding/splash.png	1200x675 atau 1600x900
Installer icon	resources/icon.ico	Untuk NSIS installer
Uninstaller icon	resources/icon.ico	Untuk NSIS uninstaller

11.1 Design Direction Icon

- Premium dark modern.
- Cinematic AI feel.
- Rounded square app icon.
- Abstract letter M combined with cinematic frame and small AI sparkle.
- Blue, violet, cyan gradient with soft glow.
- Readable at 16x16 and 32x32 taskbar size.
- No long text inside icon.
- Avoid robot face, busy filmstrip, cartoon style, excessive detail.

11.2 Prompt Desain Icon

```
Buat icon aplikasi desktop Windows untuk "MASJAVAS AI", aplikasi AI cinematic video generator.  
  
Style:  
premium dark modern, cinematic AI, rounded square app icon, abstract letter M combined with subtle film  
frame and AI sparkle, blue violet cyan gradient, soft glow, high contrast, minimal, professional SaaS  
desktop app, clean edges, readable at small taskbar size, no text, no watermark.  
  
Icon should work at:  
16x16, 32x32, 48x48, 256x256.  
  
Background:  
dark navy / black glassmorphism.  
  
Main mark:  
abstract glowing M, cinematic frame outline, small play symbol hidden subtly.  
  
Avoid:  
robot face, busy filmstrip, long text, cartoon style, excessive details.
```

11.3 Splash Window

```
Splash copy:  
"MASJAVAS AI sedang menyiapkan workspace..."  
  
Behavior:  
- tampil saat backend embedded start  
- tampil progress ringan / breathing animation  
- setelah /api/health ready, buka main window
```

- jika backend gagal start, tampilkan error ramah:
"Workspace gagal disiapkan. Coba buka ulang aplikasi."

12. Phase H - Security dan Secrets

SECURITY REQUIREMENT:

1. Jangan bundle GROKPI_API_KEY real ke app desktop.
2. Jangan masukkan server/.env production ke installer.
3. Jangan expose Authorization header ke renderer.
4. preload.js harus contextIsolation: true.
5. BrowserWindow nodeIntegration: false.
6. IPC bridge harus terbatas.
7. Jangan memberikan file system bebas ke renderer.
8. Jangan izinkan renderer menjalankan shell command.
9. Jangan log API key, Authorization header, atau full base64 payload.

MVP private:

- user dapat mengisi API key di Settings
- key disimpan lokal dengan secure storage jika memungkinkan

Production publik:

- desktop call MASJAVAS cloud backend
- cloud backend yang menyimpan secret GrokPI

13. Phase I - Package Setup

Install dependencies:

```
npm install -D electron electron-builder wait-on cross-env  
npm install get-port
```

Tambahkan scripts package.json:

```
"desktop:dev": "concurrently \"npm run server\" \"npm run dev\" \"wait-on http://localhost:5174  
http://localhost:3000/api/health && electron desktop/main.js\"",  
"desktop:build": "npm run build && electron-builder",  
"desktop:pack": "npm run build && electron-builder --dir"  
Tambahkan electron-builder config:
```

```
"build": {  
  "appId": "com.masjvas.ai",  
  "productName": "MASJAVAS AI",  
  "directories": {  
    "output": "release"  
  },  
  "files": [  
    "dist/**/*",  
    "desktop/**/*",  
    "server/**/*",  
    "package.json"  
  ],  
  "extraResources": [  
    {  
      "from": "resources/ffmpeg/win32",  
      "to": "ffmpeg/win32"  
    },  
    {  
      "from": "resources/icon.ico",  
      "to": "icon.ico"  
    },  
    {  
      "from": "resources/branding",  
      "to": "branding"  
    }  
  ],  
  "win": {  
    "target": ["nsis"],  
    "icon": "resources/icon.ico"  
  },  
  "nsis": {  
    "oneClick": false,  
    "allowToChangeInstallationDirectory": true,  
    "createDesktopShortcut": true,  
    "createStartMenuShortcut": true,  
  }  
}
```



```

    "shortcutName": "MASJAVAS AI",
    "installerIcon": "resources/icon.ico",
    "uninstallerIcon": "resources/icon.ico",
    "installerHeaderIcon": "resources/icon.ico"
  }
}

```

14. Phase J - Desktop QA Checklist

No	Test	Expected Result
1	npm run build	Success
2	npm run desktop:pack	release unpacked created
3	Open unpacked app	No terminal required
4	Production desktop app	Does not depend on http://localhost:5174
5	Embedded backend	Starts automatically
6	Health check	Internal API health OK
7	Create project	Project JSON saved in AppData
8	Restart app	Project still visible
9	Generate references	Uploads saved in AppData/uploads
10	Generate storyboard	Hero frame and panels visible
11	Generate video	GrokPI flow works
12	Export FFmpeg final	final.mp4 created in AppData/exports
13	Delete project	Project removed safely
14	Restore backup	Project recovered from backup
15	Quit app	No orphan node process
16	Build installer	NSIS installer created
17	Install to path with spaces	App still works
18	Start Menu shortcut	Icon visible and app opens
19	Desktop shortcut	Icon crisp
20	Uninstall	Uninstaller works

15. Desktop Acceptance Criteria

- MASJAVAS AI.exe opens without terminal.
- React UI loads from dist/index.html in packaged mode.
- Embedded Express backend starts automatically on dynamic localhost port.
- Frontend service layer uses runtime API base URL.
- Project data persists in AppData/Roaming/MASJAVAS AI.
- Uploads, exports, temp, and logs persist in AppData.
- Bundled FFmpeg is used in packaged mode.
- Export final works using bundled FFmpeg.
- Icon appears in taskbar, title bar, shortcut, Start Menu, installer, and uninstaller.
- Splash screen appears while workspace starts.
- No GrokPI secret is bundled in installer.
- nodeIntegration is false and contextIsolation is true.
- Project can be saved, reopened, duplicated, renamed, deleted, and restored.
- No black screen on startup failure; show user-friendly error.
- Installer NSIS builds successfully.

16. Deliverables yang Harus Dikirim Programmer

1. implementation_plan.md untuk Desktop Windows Build.
2. Daftar file yang dibuat/diubah.
3. Screenshot atau deskripsi splash, main window, icon, installer.
4. Log npm run build.
5. Log npm run desktop:pack.
6. Log app packaged running dari release unpacked.
7. Lokasi AppData runtime folders.
8. Hasil FFmpeg export dari packaged app.
9. Hasil QA checklist desktop.
10. Known issues dan rekomendasi fase berikutnya.

17. Laporan Akhir yang Diminta

SETELAH SELESAI, KIRIM LAPORAN:

1. Desktop architecture final.
2. Dynamic port behavior.
3. Runtime storage path.
4. FFmpeg bundled path.
5. Security handling untuk API key.
6. Icon dan branding assets yang sudah dipasang.
7. Installer/packaging result.
8. QA result:
 - build
 - pack
 - open app
 - create/reopen project
 - generate/export
 - quit cleanup
9. Known issues.
10. Rekomendasi next phase.

18. Catatan Final

Baseline MVP web jangan diubah besar-besaran kecuali diperlukan untuk desktop compatibility. Fokus fase ini adalah packaging, runtime path, security, icon/branding, dan reliability di Windows.

Implementasi harus tetap mengikuti prinsip MASJAVAS: sistem modular, production-ready, tidak generik, dan ramah user pemula.

End of implementation prompt - MASJAVAS AI Desktop Windows Build